

```

# =====
# 0. WORKING DIRECTORY
# =====
# When run interactively in RStudio, set the working directory
# to the folder that contains this script so that all relative
# paths resolve correctly:
# source("API Data Pull_Copy.R")
#   "data/florida_master.rds"
#   "data/predictions_2024.csv"
# This block is safely skipped when the script is run via
# Rscript from a terminal already at the project root.

if (requireNamespace("rstudioapi", quietly = TRUE) && rstudioapi::isAvailable()) {
  setwd(dirname(rstudioapi::getActiveDocumentContext()$path))
}

library(tidyverse)
library(tidymodels)
library(readxl)      # read_xlsx() for Section 12 BLS comparison
library(glmnet)      # Lasso / Ridge engine
library(ranger)      # Random Forest engine
library(xgboost)     # XGBoost engine
library(doParallel)  # parallel backend for tune_grid()

# =====
# PARALLEL SETUP
# =====
# Register a doParallel PSOCK cluster so tune_grid() distributes
# resample x grid-candidate evaluations across cores automatically.
# Uses base-R PSOCK workers (no future/doFuture) – safe in RStudio
# and works on Mac, Windows, and Linux.
# Capped at 4 workers to avoid RAM exhaustion on MacBook Air.
# IMPORTANT: requires a FRESH R session (Session → Restart R) to avoid
# interference from any previously loaded doFuture/future packages.

n_cores <- max(1L, min(4L, parallel::detectCores(logical = FALSE) - 1L))
cl      <- makeCluster(n_cores)
registerDoParallel(cl)
on.exit(stopCluster(cl), add = TRUE)  # clean up workers even if the script errors mid-
run
message(sprintf("Parallel backend: %d doParallel workers", n_cores))
message(" >> tune_grid() runs will be silent while fitting (PSOCK workers don't relay
stdout)")
message(" >> Per-model [i/5] timing messages still appear from the main process")

# =====
# 1. LOAD DATA
# =====
# Source the API pull script to obtain florida_master.
# Result is cached as data/florida_master.rds so subsequent runs
# skip the slow API calls – delete the .rds to force a refresh.

cache_path <- "data/florida_master.rds"

if (file.exists(cache_path)) {
  message("Loading cached florida_master from ", cache_path)
  florida_master <- readRDS(cache_path)
} else {
  message("Cache not found – sourcing API Data Pull_Copy.R (may take several minutes)
...")
  t0_api <- proc.time()
  source("API Data Pull_Copy.R")

```

```

message(sprintf("API pull complete (%.0f sec). Saving cache ...",
               (proc.time() - t0_api)[["elapsed"]]))
saveRDS(florida_master, cache_path)
message("Saved to ", cache_path)
}

# =====
# 2. BUILD MODELING DATA FRAME
# =====
# Keep only numeric predictors (no occupation/industry ID columns).
# Filter to 2016+ per the train/test design (train 2013–2016, test 2024).
# drop_na() ensures all five models train on identical observations.
#
# Note: BEA aggregate variables (nominal_gsp, real_gsp, cnp, lfpr, etc.)
# vary only by Year, while qcew and BEA industry variables vary by
# Year x NAICS. Year is retained as a numeric predictor to capture
# macroeconomic time trends, consistent with Project Script.R.

model_df <- florida_master %>%
  arrange(Year) %>%
  group_by(occ_code, naics_2digit) %>%
  mutate(
    # Employment history – lags 1–8 back to 2015
    # lag1 is available for all years >= 2016; deeper lags accumulate as years pass.
    # NAs for shorter histories are imputed in the recipe via step_impute_median().
    oews_tot_emp_lag1 = lag(oews_tot_emp, 1),
    oews_tot_emp_lag2 = lag(oews_tot_emp, 2),
    oews_tot_emp_lag3 = lag(oews_tot_emp, 3),
    oews_tot_emp_lag4 = lag(oews_tot_emp, 4),
    oews_tot_emp_lag5 = lag(oews_tot_emp, 5),
    oews_tot_emp_lag6 = lag(oews_tot_emp, 6),
    oews_tot_emp_lag7 = lag(oews_tot_emp, 7),
    oews_tot_emp_lag8 = lag(oews_tot_emp, 8),
    # OEWS wages – lag 1 only (same OEWS file as target; contemporaneous = leakage)
    oews_h_mean_lag1 = lag(oews_h_mean),
    oews_a_mean_lag1 = lag(oews_a_mean),
    # Macro/pay variables – lag 1 (per design spec; not available at prediction time)
    qcew_wtd_avg_pay_lag1 = lag(qcew_wtd_avg_pay),
    nominal_gsp_lag1 = lag(nominal_gsp),
    real_gsp_lag1 = lag(real_gsp),
    pce_lag1 = lag(pce),
    government_gdp_lag1 = lag(government_gdp),
    construction_gdp_lag1 = lag(construction_gdp),
    gdp_deflator_lag1 = lag(gdp_deflator)
  ) %>%
  ungroup() %>%
  filter(Year >= 2013) %>% # filter AFTER lags so 2014 rows get 2013 values; lag-
# 1 of 2013 is NA, imputed in recipe
  select(
    occ_code, naics_2digit, # ID columns – NOT predictors (see recipe)
    oews_tot_emp, # target
    Year,
    oews_tot_emp_lag1, # t–1 employment (most recent; occupation identity)
    oews_tot_emp_lag2, # t–2 employment
    oews_tot_emp_lag3, # t–3 employment
    oews_tot_emp_lag4, # t–4 employment
    oews_tot_emp_lag5, # t–5 employment
    oews_tot_emp_lag6, # t–6 employment
    oews_tot_emp_lag7, # t–7 employment
    oews_tot_emp_lag8, # t–8 employment
    oews_h_mean_lag1, # t–1 hourly wage
    oews_a_mean_lag1, # t–1 annual wage
    avg_weekly_hours, # CES (different source; contemporaneous OK)
    qcew_total_emp, # QCEW employment (different source)
  )

```

```

qcew_wtd_avg_pay_lag1,      # t-1 avg annual pay
real_gdp_industry,          # BEA industry GDP (different source)
nominal_gdp_industry,       # BEA industry GDP (different source)
cnp,                        # LAUS pop (different source)
lfpr,                       # LAUS LFPR (different source)
labor_force,               # LAUS labor force (different source)
nominal_gsp_lag1,          # t-1 nominal GSP
real_gsp_lag1,             # t-1 real GSP
pce_lag1,                  # t-1 PCE
government_gdp_lag1,       # t-1 government GDP
construction_gdp_lag1,     # t-1 construction GDP
total_employment,          # BEA employment (different source)
gdp_deflator_lag1          # t-1 GDP deflator
)
# NAs in deeper lag columns handled by step_impute_median() in the recipe

cat("Total modeling rows:", nrow(model_df), "\n")

# =====
# 3. TIME-BASED SPLIT
# =====
# Train: 2013 – 2016   (4 years; lag-1 of 2013 imputed; aligns with STWIND base year)
# Test:  2024          (held-out year for final evaluation)

train_data <- model_df %>% filter(Year <= 2016)
test_data  <- model_df %>% filter(Year == 2024)

cat("Training rows:", nrow(train_data), "\n")
cat("Testing rows: ", nrow(test_data), "\n")

stopifnot(
  # target must be complete in both splits
  "NAs in oews_tot_emp (train)" = sum(is.na(train_data$oews_tot_emp)) == 0,
  "NAs in oews_tot_emp (test)"  = sum(is.na(test_data$oews_tot_emp)) == 0,
  "No 2024 rows in test_data"  = nrow(test_data) > 0
)

# NAs are expected in multiple columns and are all handled by
step_impute_median(all_predictors()) in the recipe:
# lag1-lag8 are NA for occ x NAICS combos that first appeared after 2015
# lag2-lag8 are also NA for early training years that predate the available history
# oews_h_mean_lag1, oews_a_mean_lag1, qcew_wtd_avg_pay_lag1 follow the same pattern
message(sprintf(
  "Lag NAs in train – lag1: %d lag2: %d lag4: %d lag8: %d (all imputed in recipe)",
  sum(is.na(train_data$oews_tot_emp_lag1)),
  sum(is.na(train_data$oews_tot_emp_lag2)),
  sum(is.na(train_data$oews_tot_emp_lag4)),
  sum(is.na(train_data$oews_tot_emp_lag8))
))

# =====
# 4. PREPROCESSING RECIPE
# =====
# step_zv:      remove any zero-variance predictors
# step_normalize: standardize all numeric predictors to mean=0, sd=1
#               (required for Lasso/Ridge; ensures fair comparison
#               of penalty magnitudes across models)

base_recipe <- recipe(oews_tot_emp ~ ., data = train_data) %>%
  update_role(occ_code, naics_2digit, new_role = "ID") %>% # retain for export; exclude
  # from model features
  step_impute_median(all_predictors()) %>% # fill NAs in any predictor
  # (lag cols, missing BEA/QCEW rows, new occ combos)

```

```

step_zv(all_predictors()) %>%
step_normalize(all_numeric_predictors())

# =====
# 5. LEAVE-ONE-YEAR-OUT CROSS-VALIDATION
# =====
# Groups folds by Year so each fold holds out one complete year.
# With 4 training years (2013–2016) this gives 4 folds.
# This matches the temporal CV strategy in Project Script.R and
# prevents data from the same year appearing in both analysis and
# assessment sets.

train_folds <- group_vfold_cv(train_data, group = Year)

# =====
# 6. MODEL SPECIFICATIONS
# =====

# OLS -- ordinary least squares, no hyperparameters
spec_ols <- linear_reg() %>%
  set_engine("lm") %>%
  set_mode("regression")

# Lasso -- L1 regularization; penalty shrinks some coefficients to exactly 0
spec_lasso <- linear_reg(penalty = tune(), mixture = 1) %>%
  set_engine("glmnet") %>%
  set_mode("regression")

# Ridge -- L2 regularization; penalty shrinks all coefficients toward 0
spec_ridge <- linear_reg(penalty = tune(), mixture = 0) %>%
  set_engine("glmnet") %>%
  set_mode("regression")

# Random Forest -- ensemble of decorrelated decision trees
spec_rf <- rand_forest(
  mtry = tune(), # predictors sampled at each split
  trees = tune(), # number of trees
  min_n = tune() # minimum node size
) %>%
  set_engine("ranger", importance = "impurity", num.threads = 1) %>%
  # num.threads = 1: prevents nested parallelism – tune_grid() already
  # runs multiple ranger fits simultaneously via doParallel workers
  set_mode("regression")

# XGBoost -- gradient-boosted trees
spec_xgb <- boost_tree(
  trees = tune(),
  tree_depth = tune(),
  min_n = tune(),
  learn_rate = tune(),
  loss_reduction = tune()
) %>%
  set_engine("xgboost", nthread = 1) %>%
  # nthread = 1: prevents nested parallelism – same reason as ranger above
  set_mode("regression")

# =====
# 7. TUNING GRIDS
# =====
# Lasso/Ridge: 1-D regular grid over the penalty parameter
# RF/XGBoost: Latin hypercube for efficient coverage of the
# multi-dimensional hyperparameter space

```

```

grids <- list(
  ols = NULL, # no tuning
  lasso = grid_regular(penalty(), levels = 20),
  ridge = grid_regular(penalty(), levels = 20),
  rf = grid_latin_hypercube(
    mtry(range = c(2L, 26L)),
    trees(),
    min_n(),
    size = 20
  ),
  xgb = grid_latin_hypercube(
    trees(),
    tree_depth(),
    min_n(),
    learn_rate(),
    loss_reduction(),
    size = 25
  )
)

# =====
# 8. fit_one() HELPER
# =====
# Adapted from Project Script.R.
# - If grid is NULL (OLS): fit directly on training data.
# - Otherwise: tune via CV, select best params by RMSE,
#   finalize the workflow, and fit on the full training set.

fit_one <- function(wf, grid, folds, train) {
  if (is.null(grid)) {
    fit(wf, data = train)
  } else {
    tuned <- tune_grid(
      wf,
      resamples = folds,
      grid = grid,
      metrics = metric_set(rmse, mae, rsq),
      control = control_grid(
        verbose = FALSE, # doParallel PSOCK workers don't relay stdout to console;
                          # verbose=TRUE would silently discard all output
        allow_par = TRUE, # use the registered doParallel cluster
        pkgs = c("tidymodels", "glmnet", "ranger", "xgboost")
      )
    )
    best <- select_best(tuned, metric = "rmse")
    finalize_workflow(wf, best) %>% fit(data = train)
  }
}

# =====
# 9. BUILD WORKFLOWS & FIT ALL MODELS
# =====

wf_list <- list(
  ols = workflow() %>% add_recipe(base_recipe) %>% add_model(spec_ols),
  lasso = workflow() %>% add_recipe(base_recipe) %>% add_model(spec_lasso),
  ridge = workflow() %>% add_recipe(base_recipe) %>% add_model(spec_ridge),
  rf = workflow() %>% add_recipe(base_recipe) %>% add_model(spec_rf),
  xgb = workflow() %>% add_recipe(base_recipe) %>% add_model(spec_xgb)
)

set.seed(123)

```

```

t_total <- proc.time()
message(sprintf("\n=== Fitting %d models - %s ===",
               length(wf_list), format(Sys.time(), "%H:%M:%S")))

fitted_models <- list()
for (nm in names(wf_list)) {
  i <- which(names(wf_list) == nm)
  message(sprintf("\n[%d/%d] Starting %-6s (%s)",
                 i, length(wf_list), nm, format(Sys.time(), "%H:%M:%S")))
  t0 <- proc.time()
  fitted_models[[nm]] <- fit_one(wf_list[[nm]], grids[[nm]], train_folds, train_data)
  message(sprintf("[%d/%d] Done %-6s (%.0f sec)",
                 i, length(wf_list), nm, (proc.time() - t0)[["elapsed"]]))
}

message(sprintf("\nAll models fitted. Total elapsed: %.0f sec (%s)\n",
               (proc.time() - t_total)[["elapsed"]],
               format(Sys.time(), "%H:%M:%S")))

# =====
# 10. EVALUATE ON TEST SET (Year == 2024)
# =====

eval_metrics <- metric_set(rmse, mae, rsq)

test_metrics <- imap_dfr(fitted_models, \(m, nm) {
  augment(m, test_data) %>%
    eval_metrics(truth = oews_tot_emp, estimate = .pred) %>%
    mutate(model = nm)
}) %>%
  select(model, .metric, .estimate) %>%
  arrange(.metric, .estimate)

# Wide comparison table -- one row per model, sorted by RMSE
results_wide <- test_metrics %>%
  pivot_wider(names_from = .metric, values_from = .estimate) %>%
  arrange(rmse)

cat("\n--- Model Performance on 2024 Test Set ---\n")
print(results_wide)

best <- results_wide %>% slice_min(rmse, n = 1)
cat("\nBest model by RMSE:\n")
print(best)

# =====
# 11. EXPORT PREDICTIONS
# =====
# One row per model x observation; includes actuals for comparison.
# Follows the predictions_2024.csv pattern from Project Script.R.

predictions_2024 <- imap_dfr(fitted_models, \(m, nm) {
  augment(m, test_data) %>%
    mutate(model = nm) %>%
    select(Year, occ_code, naics_2digit, .pred, oews_tot_emp, model)
})

write_csv(predictions_2024, "data/predictions_2024.csv")
message("Predictions written to data/predictions_2024.csv")

# =====
# 12. COMPARE PREDICTIONS TO RAW 2024 BLS ACTUALS

```

```

# =====
# Load oes_research_2024_allsectors.xlsx directly from Data 2 and apply
# the same cleaning logic used in API Data Pull_Copy.R (coalesce handles
# the column-name differences across file vintages). Aggregate to the
# same occ_code x naics_2digit grain as florida_master, then join to the
# predictions and compute per-model error metrics.

message("Loading 2024 BLS OEWS file for comparison ...")

oes_2024_raw <- read_xlsx(
  "Data 2/oes_research_2024_allsectors.xlsx",
  guess_max = 10000
) %>%
  mutate(across(everything(), as.character)) %>%
  rename_with(tolower) # normalise to lowercase so names work regardless of file vintage

oes_2024_actual <- oes_2024_raw %>%
  filter(area_title == "Florida",
         o_group == "detailed") %>%
  mutate(
    naics_2digit = case_when(
      is.na(naics) | naics == "000000" ~ NA_character_,
      TRUE ~ substr(naics, 1, 2)
    ),
    tot_emp_num = suppressWarnings(as.numeric(tot_emp))
  ) %>%
  filter(!is.na(naics_2digit)) %>%
  group_by(occ_code, naics_2digit) %>%
  summarise(
    occ_title = first(occ_title),
    actual_2024 = sum(tot_emp_num, na.rm = TRUE),
    .groups = "drop"
  )

comparison_2024 <- predictions_2024 %>%
  left_join(oes_2024_actual, by = c("occ_code", "naics_2digit")) %>%
  mutate(
    error = .pred - actual_2024,
    abs_err = abs(error),
    pct_err = abs_err / actual_2024 * 100
  )

comparison_metrics <- comparison_2024 %>%
  filter(actual_2024 > 0) %>% # drop suppressed/zero-actual cells (pct_err = Inf when
actual = 0);
# also removes NA actuals (NA > 0 == NA == FALSE in

filter()
  group_by(model) %>%
  summarise(
    n = sum(!is.na(actual_2024)),
    RMSE = sqrt(mean(error^2, na.rm = TRUE)),
    MAE = mean(abs_err, na.rm = TRUE),
    MAPE = mean(pct_err, na.rm = TRUE),
    R2 = cor(.pred, actual_2024, use = "complete.obs")^2,
    .groups = "drop"
  ) %>%
  arrange(RMSE)

cat("\n--- Prediction Accuracy vs 2024 BLS Actuals ---\n")
print(comparison_metrics)

write_csv(comparison_2024, "data/comparison_2024.csv")
message("Detailed comparison written to data/comparison_2024.csv")

```

```
# =====  
# CLEANUP  
# =====  
stopCluster(cl)    # shut down worker processes cleanly  
registerDoSEQ()    # restore sequential execution  
message("Parallel backend released.")
```